

LAUNCH26

RELIC RING PROTOCOL

Windows Installation, Dependencies and Run Guide

Go + Wails v2 + React/TypeScript + Docker Compose

Prepared for the distributed Zeta-26 desktop simulator

Document version 1.0 | 27 June 2026

NATIVE DESKTOP

Wails + WebView2

DISTRIBUTED NODES

Six Go services

CONTAINER NETWORK

Docker Compose

A Quick Start Summary

What must be installed, what is already installed, and the correct start order

This guide contains every system tool, project package, verification command, and startup command required to run the Relic Ring Protocol on Windows 11. It is based on the project's go.mod, frontend/package.json, Wails configuration, Dockerfiles, Compose file, and your successful Wails Doctor result.

✓ Your current machine is already ready for Wails development

Verified from your Wails Doctor screenshot: Windows 11, Go 1.26.4, Wails 2.12.0, Node.js 22.13.1, npm 10.9.2, and Microsoft WebView2 are installed. UPX and NSIS are optional and are not needed to run the application.

The only major runtime tool that may still need installation is Docker Desktop with Docker Compose and WSL 2. Git and Visual Studio Code should also be available for normal development.

Component	Needed to run?	How it is installed
Visual Studio Code	Recommended	Windows installer
Git	Recommended	Git for Windows installer
Go 1.26.x	Required	Windows x64 MSI
Node.js + npm	Required	Node.js LTS installer
Wails CLI v2.12.x	Required	go install command
Microsoft WebView2	Required by Wails	Usually preinstalled; checked by wails doctor
WSL 2	Required for Docker's recommended backend	wsl --install
Docker Desktop + Compose	Required for distributed planet network	Windows installer
Project Go/npm packages	Required	go mod tidy and npm install

B**Contents**

Follow the sections in order on a new Windows machine

Section	Topic
1	Installation plan and compatibility
2	Visual Studio Code
3	Git for Windows
4	Go
5	Node.js and npm
6	Wails v2 and WebView2
7	WSL 2 and hardware virtualization
8	Docker Desktop and Docker Compose
9	VS Code extensions
10	Project-specific packages
11	Install project dependencies
12	Run the full application
13	Ports and application URLs
14	Tests and production builds
15	Troubleshooting
16	Clean-machine checklist
17	Official references

Recommended installation order

VS Code -> Git -> Go -> Node.js/npm -> Wails -> WebView2 check -> WSL 2 -> Docker Desktop -> project dependencies.

1 Installation Plan and Compatibility

Use versions that match the project manifests and Docker build files

The project module declares Go 1.23.0, while both Dockerfiles build with golang:1.26-alpine. For the closest match between local development and Docker, use Go 1.26.x. The official Go download page currently provides Go 1.26.4 for Windows x64. The frontend uses Vite 6, React 18, and TypeScript 5.7. Your installed Node.js 22.13.1 has already successfully run the Wails application, so no Node reinstall is necessary. For a clean new machine, use a supported Node.js LTS release.

Tool	Project requirement	Recommended / verified
Windows	Wails-supported Windows 10/11	Windows 11 x64
Go	go.mod: 1.23.0 minimum; Docker: 1.26	Go 1.26.4
Node.js	Needed for npm and Vite	Node 22 LTS or 24 LTS
npm	Installed with Node.js	npm 10+ or 11+
Wails	go.mod dependency v2.12.0	Wails CLI v2.12.0
WebView2	Required on Windows by Wails	Installed and verified
Docker	Compose file and seven containers	Docker Desktop with Compose

! Do not install Wails v3 for this repository

This project imports github.com/wailsapp/wails/v2 and uses the Wails v2 configuration format. Install the v2 CLI command shown in this guide.

Internet access is required the first time you install system tools, download Go modules, download npm packages, and build Docker images.

2 Install Visual Studio Code

Editor, integrated terminal, debugging, extensions, and project tasks

1. Download the Windows User Installer from the official Visual Studio Code website.
2. Run the installer and keep Add to PATH enabled.
3. Enable Open with Code for files and folders if offered.
4. Close and reopen all terminals after installation.

Verify:

```
code --version  
code .
```

Open the extracted project folder using File -> Open Folder, or use:

```
cd "D:\Team-AlchemiX-LAUNCH-26-project-structure\relic-ring-protocol\relic-ring-protocol"  
code .
```

Extracted ZIP folders are acceptable

You can install tools while the terminal is inside the extracted project folder. System tools are installed for Windows, not inside the project directory. Keeping the project in a short, clear path will make Docker and terminal commands easier.

Official download:

[Visual Studio Code for Windows](#)

3 Install Git for Windows

Repository cloning, branches, commits, pull requests, and team collaboration

5. Download the 64-bit Git for Windows installer.
6. Choose Visual Studio Code as the default editor when asked.
7. Select the option that allows Git from the command line and third-party software.
8. Keep the recommended line-ending setting unless your team has another standard.
9. Restart VS Code after installation.

Verify and configure:

```
git --version
git config --global user.name "Your Name"
git config --global user.email "your-github-email@example.com"
git config --global --list
```

Git is not required merely to launch an already-extracted copy, but it is strongly recommended because the project is designed for a team GitHub workflow.

[Official Git for Windows download](#)

4 Install Go

Core language, compiler, tests, orchestrator, planet-node services, and Wails backend

10. Download the Windows x64 MSI installer from the official Go downloads page.
11. Use the default installation location, normally C:\Program Files\Go.
12. Complete the installer and fully restart VS Code.

Verify:

```
go version
where.exe go
go env GOPATH
```

✓ Recommended version for this repository

Use Go 1.26.x. Your machine already reports Go 1.26.4, which matches the Docker build major/minor line and is suitable for the project.

Go installs command-line programs, including Wails, into the bin folder under GOPATH. On Windows this is usually:

```
C:\Users\<YOUR_USERNAME>\go\bin
```

Official sources:

[Go downloads](#) | [Go installation instructions](#)

5

Install Node.js and npm

Builds the React/TypeScript frontend and runs Vite

13. Download the Windows x64 Installer for a supported Node.js LTS release.
14. Keep npm and Add to PATH selected.
15. Complete the installation and restart VS Code.

Verify:

```
node --version
npm --version
where.exe node
where.exe npm
```

✓ Your existing version is already working

Your Wails Doctor result shows Node.js 22.13.1 and npm 10.9.2. The desktop application has run successfully, so you do not need to replace them only to follow this guide.

For a new installation, the official Node.js page currently lists Node.js 24.18.0 as the latest LTS release. npm is included with Node.js.

[Official Node.js download](#)

6 Install Wails v2 and Verify WebView2

Creates the native Windows desktop shell around the Go and React application

Install the Wails v2 CLI after Go and Node.js are working:

```
go install github.com/wailsapp/wails/v2/cmd/wails@latest
```

Add the Go binary folder to the current PowerShell session:

```
$env:Path += ";$(go env GOPATH)\bin"
```

Verify Wails and all Windows dependencies:

```
wails version
wails doctor
```

✓ Expected result

Wails Doctor should end with SUCCESS: Your system is ready for Wails development. Your screenshot already confirms Wails 2.12.0, Go, Node.js, npm, and WebView2 are installed.

If the wails command disappears after restarting, add this folder permanently to the User PATH:

```
$goBin = "$(go env GOPATH)\bin"
$userPath = [Environment]::GetEnvironmentVariable("Path", "User")
if (($userPath -split ";") -notcontains $goBin) {
    [Environment]::SetEnvironmentVariable("Path", "$userPath;$goBin", "User")
}
```

WebView2 is the Windows browser runtime used inside the native desktop window. It is often already installed. Wails Doctor verifies it.

Optional packages:

- UPX - compresses a built executable; not needed to develop or run the app.
- NSIS - creates a Windows installer; not needed to develop or run the app.

[Wails v2 installation](#) | [Microsoft WebView2 Runtime](#)

7

Install WSL 2 and Check Virtualization

Recommended Windows backend for Docker Desktop Linux containers

Open PowerShell as Administrator and run:

```
wsl --install
```

Restart Windows when requested. Then verify and update:

```
wsl --update  
wsl --set-default-version 2  
wsl --version  
wsl --list --verbose
```

! Virtualization must be enabled

Open Task Manager -> Performance -> CPU and confirm Virtualization: Enabled. If disabled, enable Intel VT-x, AMD-V, or SVM in the computer's BIOS/UEFI.

WSL is not required by the Go or Wails code itself. It is needed for Docker Desktop's recommended Linux-container backend on Windows.

[Official Microsoft WSL installation guide](#)

8

Install Docker Desktop and Docker Compose

Runs the six planet nodes and the orchestrator as independent services

16. Install WSL 2 first and restart Windows.
17. Download Docker Desktop for Windows x86-64.
18. During installation, select the WSL 2 based engine when available.
19. Open Docker Desktop and wait until the engine is running.
20. In Docker settings, confirm Use the WSL 2 based engine is enabled.

Verify:

```
docker --version
docker compose version
docker run hello-world
```

The project Compose file creates these seven services:

Service	Port	Purpose
aegis	8101	Planet node
boreas	8102	Planet node
dawn	8103	Planet node
elysium	8104	Planet node
fenix	8105	Planet node
caelum	8106	Planet node
orchestrator	8080	Routing, health, simulation, API

First Docker build takes longer

Docker must download the Go and Alpine base images, download Go modules, compile two binaries, and start seven containers. Later builds are usually faster because Docker uses its cache.

[Official Docker Desktop for Windows guide](#)

9

Install Recommended VS Code Extensions

Language intelligence, formatting, Docker control, and frontend support

Extension	Publisher / purpose	Required?
Go	Go Team at Google - gopls, tests, formatting, debugging	Recommended
Docker	Microsoft - containers, images, Compose files	Recommended
ESLint	Microsoft - JavaScript/TypeScript linting	Recommended
Prettier	Prettier - frontend formatting	Recommended
YAML	Red Hat - Compose YAML editing	Useful
GitLens	Git history and collaboration	Optional
REST Client / Thunder Client	API testing inside VS Code	Optional
Error Lens	Inline compiler and lint errors	Optional

The repository also contains `.vscode/extensions.json` and `.vscode/tasks.json`. VS Code may offer to install recommended extensions automatically.

When the Go extension asks to install Go tools such as `gopls`, `dlv`, `goimports`, and `staticcheck`, choose **Install All**.

10 Project-Specific Packages

These are installed automatically from the repository manifests

Do not manually download the libraries below. Run go mod tidy and npm install from the correct folders.

Go dependency from go.mod:

Module	Version	Role
github.com/wailsapp/wails/v2	v2.12.0	Desktop application framework and Go bindings

Frontend packages from frontend/package.json:

Package	Declared version	Role
react	^18.3.1	User interface
react-dom	^18.3.1	React browser renderer
vite	^6.1.0	Frontend development/build server
typescript	^5.7.3	Type-safe frontend code
@vitejs/plugin-react	^4.3.4	React support for Vite
lucide-react	^0.468.0	Interface icons
@types/react	^18.3.18	React TypeScript definitions
@types/react-dom	^18.3.5	React DOM TypeScript definitions

Docker base images pulled automatically during the build:

Image	Used for
golang:1.26-alpine	Compiling planet-node and orchestrator binaries
alpine:3.22	Small runtime image for the compiled Go services

✓ No database installation is required

The current Relic Ring implementation stores the network and simulation state in memory and reads the universe from JSON. PostgreSQL, MySQL, MongoDB, Redis, and similar services are not required.

11 Install the Project Dependencies

Run these commands once after opening the extracted repository

Open the VS Code terminal in the project root:

```
cd "D:\Team-AlchemiX-LAUNCH-26-project-structure\relic-ring-protocol\relic-ring-protocol"
```

Install and normalize Go modules:

```
go mod tidy
```

Install frontend packages:

```
cd frontend  
npm install  
cd ..
```

Run initial verification:

```
go test ./...  
go vet ./...  
cd frontend  
npm run build  
cd ..
```

Optional setup script

The repository includes scripts/setup-windows.ps1. It checks Go, downloads Go modules, installs frontend packages, and runs Go tests.

```
Set-ExecutionPolicy -Scope Process Bypass  
.\scripts\setup-windows.ps1
```

If npm install creates package-lock.json, commit it so every team member installs the same resolved dependency versions.

12 Run the Full Application

Docker network first, then Wails desktop in a second terminal

Terminal 1 - start the distributed network:

```
docker compose -f deployments/docker/compose.yaml up --build
```

Keep Terminal 1 running. Open Terminal 2 and check container status:

```
docker compose -f deployments/docker/compose.yaml ps
```

Wait until all six planets are healthy and the orchestrator is running. Then start the desktop app:

```
wails dev
```

✓ Correct startup order

1) Start Docker Desktop. 2) Start Docker Compose. 3) Wait for healthy services. 4) Run wails dev. The desktop app connects to the orchestrator at <http://localhost:8080>.

Stop the complete network:

```
docker compose -f deployments/docker/compose.yaml down
```

Chaos test - stop and restore Dawn:

```
docker compose -f deployments/docker/compose.yaml stop dawn
docker compose -f deployments/docker/compose.yaml start dawn
```

Manual backend alternative without Docker: start the six planet-node commands on ports 8101-8106 in separate terminals, then run `go run ./cmd/orchestrator`. Docker is strongly preferred for the final demonstration because failures are easier to prove.

13

Ports, URLs, and the Native Desktop Window

Know which address belongs to which part of the system

Address / port	Purpose	Open directly?
Native Wails window	Main control centre and normal user interface	Yes - recommended
http://localhost:8080	Go orchestrator REST API and events	API, not the main UI
http://localhost:5173	Vite frontend development server/assets	May appear blank outside Wails
8101-8106	Independent planet HTTP services	Health/API checks only

! Why localhost:5173 may be blank

The interface calls Go methods through the Wails runtime bridge. The native WebView2 window provides that bridge. Opening Vite directly in a normal browser may not provide the required Wails bindings, so the page can be blank even while the desktop application works correctly.

Use the native window launched by wails dev for normal testing.

Useful API checks:

```
Invoke-RestMethod http://localhost:8080/api/universe
$body = @{ origin_id='Aegis'; destination_id='Caelum'; payload='Hello world' } | ConvertTo-Json
Invoke-RestMethod -Method Post -Uri http://localhost:8080/api/transmissions `
  -ContentType 'application/json' -Body $body
```


14 Tests, Quality Checks, and Production Builds

Verify the protocol before recording the final demonstration

Backend tests and checks:

```
go test ./...
go test -race ./...
go vet ./...
go fmt ./...
```

Frontend build:

```
cd frontend
npm run build
cd ..
```

Build the Windows desktop executable:

```
wails build
```

The production executable is normally created under:

```
build\bin\RelicRingProtocol.exe
```

Optional package audit commands:

```
cd frontend
npm outdated
npm audit
cd ..
```

! Do not update dependencies immediately before judging

First preserve the working versions in go.mod, go.sum, package.json, and package-lock.json. Test upgrades on a separate branch so a last-minute package change does not break the demo.

15 Troubleshooting Guide

Common Windows setup and startup failures

Problem	Likely cause	Fix
'go' is not recognized	Go not installed or old terminal PATH	Install Go, close all VS Code windows, reopen, run go version
'wails' is not recognized	GOPATH\bin missing from PATH	Add \$(go env GOPATH)\bin and restart terminal
Wails Doctor reports WebView2 missing	Runtime not installed	Install Microsoft WebView2 Evergreen Runtime
Docker command works but engine fails	Docker Desktop not running	Open Docker Desktop and wait for engine ready
WSL installation fails	No admin terminal or Windows restart pending	Run PowerShell as Administrator, update Windows, restart
Container is unhealthy	Port conflict or service startup error	Run docker compose logs <service>
Port 8080 already in use	Another orchestrator/process is running	Stop old process or inspect netstat -ano findstr :8080
npm install fails	Network/proxy/cache issue	Check internet/proxy; run npm cache verify and retry
go mod tidy fails	Network/proxy or certificate issue	Check internet, GOPROXY, antivirus/proxy settings
localhost:5173 is blank	Normal browser lacks Wails bridge	Use the native Wails window

Useful diagnostic commands:

```
wails doctor
docker compose -f deployments/docker/compose.yaml ps
docker compose -f deployments/docker/compose.yaml logs -f
netstat -ano | findstr :8080
netstat -ano | findstr :8101
go env
npm config list
```

PowerShell script execution policy

When Windows blocks a repository PowerShell script, use Set-ExecutionPolicy -Scope Process Bypass. This changes only the current terminal session.

16

Clean-Machine Installation Checklist

Use this list when setting up another team member's computer

☐	Windows is updated and hardware virtualization is enabled.
☐	Visual Studio Code opens and <code>code --version</code> works.
☐	Git is installed and <code>git --version</code> works.
☐	Go 1.26.x is installed and <code>go version</code> works.
☐	Node.js LTS and npm are installed.
☐	Wails v2 CLI is installed and <code>wails version</code> works.
☐	<code>wails doctor</code> reports the system is ready.
☐	Microsoft WebView2 is installed.
☐	WSL 2 is installed and <code>wsl --list --verbose</code> shows version 2.
☐	Docker Desktop is running with the WSL 2 engine.
☐	<code>docker run hello-world</code> succeeds.
☐	The project ZIP is extracted and opened at its actual root folder.
☐	<code>go mod tidy</code> completes.
☐	frontend/npm install completes.
☐	<code>go test ./...</code> passes.
☐	<code>npm run build</code> completes.
☐	Docker Compose starts six healthy planet nodes and the orchestrator.
☐	<code>wails dev</code> opens the Relic Ring Control Centre.
☐	Aegis to Boreas delivery succeeds.
☐	Aegis to Caelum selects a multi-hop route.
☐	Stopping Dawn causes the next packet to use another route.

✓ Submission readiness

Before recording, run the complete setup on a clean machine or another team member's laptop. Keep a recorded fallback demo and verify that the README contains the same commands as this guide.

17 Official References and Project Sources

Use these pages for current installers and platform-specific troubleshooting

- [\[1\] Go downloads](#)
- [\[2\] Go installation](#)
- [\[3\] Node.js downloads](#)
- [\[4\] Wails v2 installation](#)
- [\[5\] Wails production builds](#)
- [\[6\] Docker Desktop on Windows](#)
- [\[7\] Microsoft WSL installation](#)
- [\[8\] Visual Studio Code on Windows](#)
- [\[9\] Git for Windows](#)
- [\[10\] Microsoft WebView2 Runtime](#)

Project source files reviewed for this guide:

- go.mod and go.sum
- frontend/package.json
- wails.json
- deployments/docker/Dockerfile.planet
- deployments/docker/Dockerfile.orchestrator
- deployments/docker/compose.yaml
- .env.example
- Makefile
- scripts/setup-windows.ps1
- README.md

Version note

Tool versions change over time. Prefer the official installer pages above, but keep the repository's declared major versions and test every update before merging it into the team branch.

READY TO LAUNCH

Docker Compose -> Wails Desktop -> Zeta-26 Online

Run: `docker compose -f deployments/docker/compose.yaml up --build`

Then: `wails dev`